

Privacy-Preserving Natural Language Processing

RUHR
UNIVERSITÄT
BOCHUM

RUB

Lecture 6 – Local DP part 2 and Approximate DP

Prof. Dr. Ivan Habernal

June 18, 2026

www.trusthlt.org

Trustworthy Human Language Technologies Group (TrustHLT)

Ruhr University Bochum & Research Center Trustworthy Data Science and Security



CENTER FOR TRUSTWORTHY
DATA SCIENCE AND SECURITY

Recap

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP
- 4 Approximate Differential Privacy
- 5 What is this δ doing?
- 6 Gaussian mechanism
- 7 General properties of DP algorithms

What we covered so far

- For provably private data analysis we need randomized algorithms
- Central (with a trusted curator) pure $(\epsilon, 0)$ differential privacy
- Laplace mechanism: numeric queries, ℓ_1 sensitivity,
- Exponential mechanism: 'any-range' queries (arbitrary sets), utility function and its sensitivity

Using local DP in NLP

- 1 Recap
- 2 Using local DP in NLP**
- 3 Approximate DP
- 4 Approximate Differential Privacy
- 5 What is this δ doing?
- 6 Gaussian mechanism
- 7 General properties of DP algorithms

Motivation for Local DP in NLP

- Each data-owner privatizes their data point before releasing to public
- Possible use: Privacy protection of training data for classification tasks
- Assumption: n individuals each holding data-point (text document + categorical label)
- Example: Reviews for binary sentiment analysis (e.g., IMDB)
- Sensitive example: Drug reviews (review + stars)
- Goal: Train model to classify review into two or multiple classes

What are we protecting?

- Single document d (e.g., "I had really bad headaches ...")
- Associated label $y \in \{0, 1, 2\}$
- DP limits information to reconstruct true answer from noisy answer
- Interpretation 1: We know it comes from Alice $\rightarrow$ DP protects content
- Interpretation 2: Anonymous collection $\rightarrow$ DP protects linkability if text contains signature

What are we protecting?

Text vs label protection

- Protect text d (Sensitive content like drug use) or protect label y (Sensitive rating)?
- If text reveals label (e.g., bad review text + 0 stars), protecting label might be useless
- Or protect text only? (Given 0-star label, we know experience was horrible)
- Usually dependent → Better protect entire data-point

Using local DP in NLP

Privatizing latent representation

Privatizing latent representation

- Classifier expects feature vector $x \in \mathbb{R}^j$ and label vector $y \in \mathbb{R}^k$
- Alice turns document d into x , label into one-hot y
- Privatize both with DP locally before releasing

Binary vectors: Bag of Words

Standard NLP task: Feature extraction / representation learning

- Baseline: *Bag of words* with vocabulary V
- Feature vector dimension $j = |V|$
- Options:
 - Raw counts: $x \in \{0, 1, 2, \dots\}^j$
 - Presence/absence: $x \in \{0, 1\}^j$
- Label y : Single bit ($y \in \{0, 1\}$) or one-hot ($y \in \{0, 1\}^k$)
- Privatize $x \rightarrow \hat{x}$ and $y \rightarrow \hat{y}$ using RR
- Split budget ε into ε_1 (features) and ε_2 (label)
- Or spend $\varepsilon_i = \varepsilon / (j + k)$ on each bit

Crucial problem: Dimensionality $j = |V|$

More features (larger V) $\rightarrow$ smaller ε_i per bit $\rightarrow$ More noise introduced to privatized vector

- Example 1: $\varepsilon = 2$, 1 bit $\rightarrow$ flip prob ≈ 0.1192 , looks OK
- Example 2: $\varepsilon = 2$, 100 bits $\rightarrow \varepsilon_i = 0.02 \rightarrow$ flip prob ≈ 0.4950
 - Result: Each bit almost completely random
 - Chances to train meaningful classifier look almost impossible
- Solution in literature: Drastically increase ε (e.g., $\varepsilon = 200$)
- Consequence: Effectively non-private

Problem with large ε

- Question: What's wrong with $\varepsilon = 200$?
- Sensitive question: 'Did you take illegal drugs?'
- Privatize with randomized response $\varepsilon = 200$
- Probability of revealing truth:

$$\frac{1}{1 + \exp(-200)} \approx 0.\underbrace{9999 \dots 9999}_{86 \text{ times}}891610$$

- Standard calculator shows 1.0
- No one willing to participate in such survey

Real-valued vectors: Embeddings

- Deep neural networks produce useful learned representations
- Turn text into high-dimensional embeddings vector
- Protect text with local DP:
 - Turn document into dense embedding (e.g., local transformer)
 - Apply DP mechanism for real-valued queries (Laplace or Gaussian)
 - Share privatized vector $\hat{x}$
- Guarantees: (ε, δ) -DP

Clipping and Sensitivity

- Real-valued queries must have bounded range
- Provide mechanism with global ℓ_1 or ℓ_2 sensitivity
- Achieved by clipping embeddings to unit ball ($C = 1$)
- Clip each dimension i : $\bar{x}_i = \min\{x_i, C\}$ or $\max\{x_i, -C\}$
- ℓ_1 sensitivity: $2C \cdot j$ (Manhattan distance)
- ℓ_2 sensitivity: $2C \cdot \sqrt{j}$
- Price for privacy: Higher dimension $j \rightarrow$ higher sensitivity $\rightarrow$ more noise

Using Privatized Embeddings

- Collect privatized embedding vectors from many users
- Vectors are in latent space ('a bunch of numbers')
- Use directly as feature vectors for training private classifier
- Provided we are also given (privatized!) labels
- Question: Can we provide something more useful than noisy text embedding?

Using local DP in NLP

Rewriting texts locally with DP
guarantees

Naive Text Rewriting Method

- Work on text level directly: 'privatize' locally to readable text
- Fix vocabulary V of $|V|$ words (include **NOWORD**)
- Define utility function $u : V \times V \mapsto \mathbb{R}$
- Measure similarity (e.g., cosine similarity of static word embeddings)
- Score between -1 and 1
- Pre-compute utility values (e.g., 1.25 billion for 50,000 words)

Exponential Mechanism Setup

- Determine sensitivity Δ for exponential mechanism
- Example: $\Delta = 2$ (max difference between $u(w, z) = -1$ and $u(w, w) = 1$)
- Fix number of tokens, for example $l = 20$ (huge limitation)
- All variables (V, u, Δ, l) publicly known

Generating word-by-word

- Query Alice: "What is your word at position i ?"
- Alice uses exponential mechanism (ε_i)
- Draw similar random word from V proportional to utilities
- Answer is $(\varepsilon_i, 0)$ -DP
- Repeat for l positions
- Overall budget $\varepsilon = \sum_{i=1}^l \varepsilon_i$

DP Guarantees and Reconstruction

- Algorithm is sound: l tokens privatized by local $(\epsilon, 0)$ -DP
- Probability of reconstructing true tokens D bounded by:

$$\frac{\Pr[D|\mathcal{M}(D) = y]}{\Pr[D'|\mathcal{M}(D') = y]} \leq \exp(\epsilon) \cdot \frac{\Pr[D]}{\Pr[D']}$$

- Takes into account priors $\Pr[D]$
- D' : Any other possible sequence of twenty tokens from vocabulary V
- Mental model: Extending randomized response (1 bit) to sequence of categorical values

Using local DP in NLP

Encoder-decoder models for private
rewriting

Encoder-decoder for Private Rewriting

- Original space of sequences is extremely high-dimensional
- Discrete DP algorithms (exponential mechanism) become infeasible
- Route: Work in lower-dimensional latent space (document embeddings)
- Unlike just privatizing embedding, use privatized latent vector to generate text

Transformer-based Approach

- Early transformers (T5, BART): Encoder + Decoder
- Entire input compressed into single real-valued vector
- Assumption: Latent vector can be clipped
- Privatize latent representation with DP (Laplace or Gaussian)
- Decoder rewrites back to text
- Advantage: Fluent text (vs. word-by-word random)
- Hope: Output similar to input (utility) and differentially private

DP-BART

- Example application: DP-BART
- Experimented with locally rewriting training data
- Tasks: Movie/product reviews, intent classification
- 'Pruning' latent vector to reduce sensitivity improved privacy/accuracy trade-off
- Result: Still ended up using absurdly large ϵ values
- Exceeded 200 for most datasets

Local DP Summary

Local DP: No trusted curator needed

Enables privatize raw data and publish with plausible deniability

Randomized response: Known and used since the 1960's

Relevance to NLP: Publishing data for training (but comes with a cost)

Privatizing text practically useless with pure DP, unless absurdly high ϵ , or something wrong, or very non-standard, or all of them

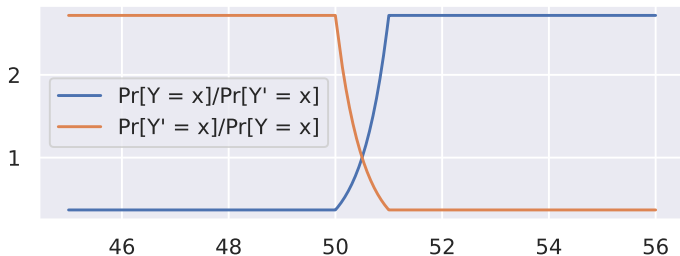
Approximate DP

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP**
- 4 Approximate Differential Privacy
- 5 What is this δ doing?
- 6 Gaussian mechanism
- 7 General properties of DP algorithms

Approximate DP

Privacy Loss Random Variable

Previously: Can we generalize it for any observed x ?



Seems like the maximum we can get is $2.718 = e = \exp(1)$

Previously: How does that relate to the maximum privacy loss?

Recall: likelihood of any output (x -axis) coming from D' as opposed to D (and vice versa)

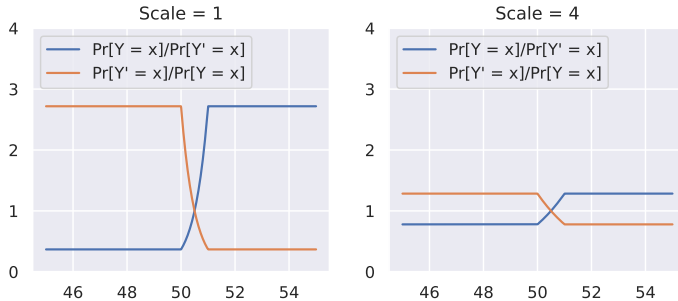


Figure 1: Privacy loss for two Laplace distributions for a counting query, varying scale b

Recall: $(\varepsilon, 0)$ differential privacy (aka. pure DP)

C. Dwork and A. Roth (2013). "**The Algorithmic Foundations of Differential Privacy**". In: *Foundations and Trends® in Theoretical Computer Science* 9.3-4, pp. 211–407, Definition 2.4

A randomized algorithm (mechanism) $\mathcal{M}$ is $(\varepsilon, 0)$ -**differentially private** if for any two neighboring datasets D, D' and any output $\mathcal{Y} \subseteq \mathcal{Z}$ this guarantee holds:

$$\Pr[\mathcal{M}(D) \in \mathcal{Y}] \leq \exp(\varepsilon) \Pr[\mathcal{M}(D') \in \mathcal{Y}]$$

We bounded our ‘privacy loss’ by ε

$$\Pr[\mathcal{M}(D) \in \mathcal{Y}] \leq \exp(\varepsilon) \Pr[\mathcal{M}(D') \in \mathcal{Y}]$$
$$\ln \left(\frac{\Pr[\mathcal{M}(D) \in \mathcal{Y}]}{\Pr[\mathcal{M}(D') \in \mathcal{Y}]} \right) \leq \varepsilon$$

What is $\mathcal{M}(D)$ (and also $\mathcal{M}(D')$)?

The private mechanism is randomized, so somewhere in the mechanism there is a random variable

- e.g., Laplace mechanism uses Laplace R.V.
- Randomized response uses Bernoulli R.V., etc.

In general, since the mechanism $\mathcal{M}(D)$ is a function of a random variable, it is also a random variable

Towards the ‘privacy loss’ random variable

$$\ln \left(\frac{\Pr[\mathcal{M}(D) \in \mathcal{Y}]}{\Pr[\mathcal{M}(D') \in \mathcal{Y}]} \right) \leq \varepsilon$$

$\mathcal{M}(D)$ (but also $\mathcal{M}(D')$) are random variables

In general, since the mechanism $\mathcal{M}(D)$ is a random variable, the entire left-hand side function $\ln \left(\frac{\Pr[\mathcal{M}(D) \in \mathcal{Y}]}{\Pr[\mathcal{M}(D') \in \mathcal{Y}]} \right)$ is again a random variable

(recall: random variables can be ‘pushed through’ functions. If X is a random variable, then $Y = g(X)$ is also a random variable. Here the g is a complicated function even including probability of X)

Step aside: You know functions of RV having similar form

If X is a discrete random variable

Expectation of X ?

$$\mathbb{E}(X) = \sum_{x \in \text{Range}(X)} x \cdot \Pr[X = x]$$

Entropy of X ? (notice lazy notation for $P[X]$ and $\sum_x$)

$$\mathbb{H}(X) = \mathbb{E} \left(\log \frac{1}{P[X]} \right) = - \sum_x x \cdot \log \Pr[X = x]$$

Step aside: You know functions of RV having similar form

KL-Divergence between X and Y (same range)?

$$\mathbb{D}(X||Y) = \mathbb{E} \left[\log \frac{P(X)}{P(Y)} \right]$$

Here we implicitly assume $\log \frac{P(X)}{P(Y)}$ is a function of X and is therefore distributed according to X

$$\begin{aligned} \mathbb{D}(X||Y) &= \mathbb{E} \left[\log \frac{P(X)}{P(Y)} \right] = \mathbb{E}[g(X)] = \sum_{x \in \text{Range}(X)} \Pr[X = x] \cdot g(x) \\ &= \sum_{x \in \text{Range}(X)} \Pr[X = x] \log \frac{\Pr[X = x]}{\Pr[Y = x]} \end{aligned}$$

Privacy Loss Random Variable

$\mathcal{M}(D)$ and $\mathcal{M}(D')$ are two random variables

The privacy loss random variable is defined as

$$\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')} = \ln \left(\frac{\Pr[\mathcal{M}(D) = t]}{\Pr[\mathcal{M}(D') = t]} \right)$$

and is distributed by drawing $t \sim \mathcal{M}(D)$

(Sanity check: You should know how to compute the expectation of the privacy loss R.V. given the previous slides)

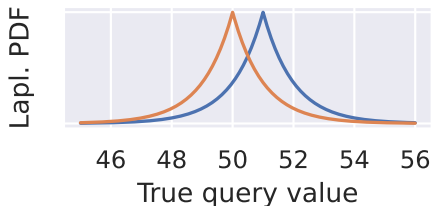
Example of Privacy Loss Random Variable

The privacy loss random variable

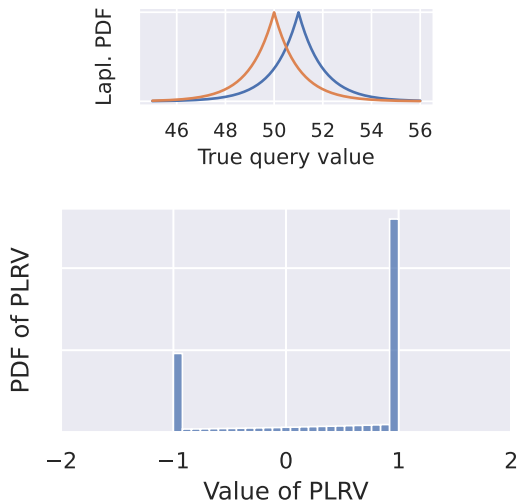
$$\mathcal{L}_{\mathcal{M}(D) \parallel \mathcal{M}(D')} = \ln \left(\frac{\Pr[\mathcal{M}(D) = t]}{\Pr[\mathcal{M}(D') = t]} \right)$$

and is distributed by drawing $t \sim \mathcal{M}(D)$

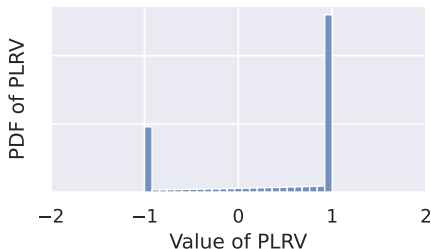
How would the distribution of $\mathcal{L}_{\mathcal{M}(D) \parallel \mathcal{M}(D')}$ would look like for the Laplace mechanism?



Example of $\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')}$ for Laplace mechanism $\varepsilon = 1$



Values of $\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')}$ are upper-bounded by ε in $(\varepsilon, 0)$ -DP



This distribution demonstrates (not a proof!) that the probability the value of $\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')}$ exceeds ε is zero

In other words

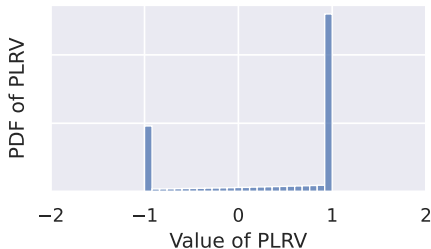
$$\Pr \left[\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')} \leq \varepsilon \right] = 1$$

Approximate Differential Privacy

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP
- 4 Approximate Differential Privacy**
- 5 What is this δ doing?
- 6 Gaussian mechanism
- 7 General properties of DP algorithms

Maybe we don't need to always ensure the bound

What if we allow to exceed ε with some small probability δ ?



In other words change $\Pr [|\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')}| \leq \varepsilon] = 1$ into

$$\Pr [\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')} \leq \varepsilon] \geq 1 - \delta$$

$$\Pr [\mathcal{L}_{\mathcal{M}(D)||\mathcal{M}(D')} > \varepsilon] < \delta \quad (\text{equivalent})$$

Formalizing approximate (ε, δ) -DP

A randomized algorithm (mechanism) $\mathcal{M}$ is (ε, δ) -**differentially private** if for any two neighboring datasets D, D' and any output $\mathcal{Y} \subseteq \mathcal{Z}$ this guarantee holds:

$$\Pr[\mathcal{M}(D) \in \mathcal{Y}] \leq \exp(\varepsilon) \Pr[\mathcal{M}(D') \in \mathcal{Y}] + \delta$$

One immediate observation: for $\delta = 0$ we get our known 'pure' DP (that's why we called it (ε, δ) -DP)

C. Dwork and A. Roth (2013). "**The Algorithmic Foundations of Differential Privacy**". In: *Foundations and Trends® in Theoretical Computer Science* 9.3-4, pp. 211–407, Definition 2.4

Formalizing approximate (ε, δ) -DP

C. Dwork and A. Roth (2013). **"The Algorithmic Foundations of Differential Privacy"**. In: *Foundations and Trends® in Theoretical Computer Science* 9.3-4, pp. 211–407, Definition 2.4

$$\Pr[\mathcal{M}(D) \in \mathcal{Y}] \leq \exp(\varepsilon) \Pr[\mathcal{M}(D') \in \mathcal{Y}] + \delta$$

This implies (but the converse is not true!)¹ that the P.L.R.V. is bounded by ε with probability $1 - \delta$

$$\Pr[\mathcal{L}_{\mathcal{M}(D) \parallel \mathcal{M}(D')} \leq \varepsilon] \geq 1 - \delta$$

¹The proof is lengthy and technical, see Dwork and Roth (2013, pp. 44–47)

What is this δ doing?

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP
- 4 Approximate Differential Privacy
- 5 What is this δ doing?**
- 6 Gaussian mechanism
- 7 General properties of DP algorithms

Extreme algorithm 1: When bad things are really bad

Our query is: Given a database of secrets, give me all rows

Name	Hospitalized in year	Age	Illegal drug use
Alice	2024	32	yes
Bob	2020	21	no
Charlie	2023	45	no
...			
Xander	2020	31	yes

Table 1: Example database D

Our goal is to have this algorithm (ϵ, δ) -DP

Given a database of secrets, give me all rows (part 1)

With probability $1 - \delta$, return completely random table

Name	Hospitalized in year	Age	Illegal drug use
Jim	2022	16	no
Dave	2011	71	yes
...			

Table 2: Example output of $\mathcal{M}(D)$ — completely random

Since the output is completely random, there would be no difference in outputs of any neighboring datasets D and D' , therefore this is perfectly private algorithm $\varepsilon = 0$

Given a database of secrets, give me all rows (part 2)

With probability δ , return the **original** dataset in full

Name	Hospitalized in year	Age	Illegal drug use
Alice	2024	32	yes
Bob	2020	21	no
...			

Table 3: Example output of $\mathcal{M}(D)$ — returning the full original D

This part of the algorithm is purely deterministic, there is no randomness, therefore this would be $\varepsilon = \infty$

Why? Remove Alice to get D' . But this algorithm is never going to return D' , so $\Pr[\mathcal{M}(D') = 0]$, which leads to $\frac{\Pr[\mathcal{M}(D)=x]}{\Pr[\mathcal{M}(D')=0]} \rightarrow \infty$

Given a database of secrets, give me all rows (part 3)

Summary of our algorithm:

- With prob. $1 - \delta$, return completely random table ($\varepsilon = 0$)
- With prob. δ , return the **original** dataset in full ($\varepsilon = \infty$)

Our algorithm is (ε, δ) -DP! (in fact $(0, \delta)$ -DP)

$$\Pr [\mathcal{L}_{\mathcal{M}(D) \parallel \mathcal{M}(D')} > \varepsilon] < \delta$$

Very bad things can happen with δ , so it should be very small! But how small?

Extreme algorithm 2: Leak just a few rows

Our query is: Given a database of secrets, give me a few rows verbatim

Name	Hospitalized in year	Age	Illegal drug use
Alice	2024	32	yes
Bob	2020	21	no
Charlie	2023	45	no
...			
Xander	2020	31	yes

Table 4: Example database D

Our goal is to have this algorithm $(0, \delta)$ -DP

Extreme algorithm 2: Leak just a few rows

Our algorithm 2:

- Iterate over all n rows
- For each row **independently**, with probability δ add this row to the output²

Name	Hospitalized in year	Age	Illegal drug use
Alice	2024	32	yes
Bob	2020	21	no
...			

Table 5: Example database D

²For instance by drawing from $\text{Ber}(\delta)$
44 Lecture 6 – Local DP part 2 and Approximate DP

Extreme algorithm 2: Leak just a few rows (part 2)

Our algorithm 2:

- Iterate over all n rows
- For each row **independently**, with probability δ add this row to the output

This has again bad consequences! Probability that at least one person will be leaked?³

$$1 - (1 - \delta)^n \approx \delta n \quad \text{for small } \delta$$

³Why? Pr. that a single person will not be leaked = $(1 - \delta)$. Pr. that no persons will be leaked = $(1 - \delta)^n$

Extreme algorithm 2: Leak just a few rows (part 3)

Probability that at least one person will be leaked?

$$1 - (1 - \delta)^n \approx \delta n \quad \text{for small } \delta$$

General recommendation

We should therefore consider

$$\delta \ll \frac{1}{n}$$

(ie. very small; typically $\delta = 1 \times 10^{-6}$, aka
'cryptographically' small)

Gaussian mechanism

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP
- 4 Approximate Differential Privacy
- 5 What is this δ doing?
- 6 Gaussian mechanism**
- 7 General properties of DP algorithms

ℓ_2 sensitivity

Similar to ℓ_1 sensitivity of the query

ℓ_2 sensitivity

The ℓ_2 -sensitivity of a function $f : D \rightarrow \mathbb{R}^k$:

$$\Delta_2 f = \max_{D, D'} \|f(D) - f(D')\|_2$$

Gaussian (Normal) random variable

The density (PDF) of a general univariate normal distribution $\mathcal{N}(\mu, \sigma^2)$

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \cdot \left(\frac{x - \mu}{\sigma}\right)^2\right)$$

Gaussian mechanism

Function (numeric query) $f : D \rightarrow \mathbb{R}^k$:

Very important constraints on ε !

For $\varepsilon \in (0, 1)$ and $\delta > 0$

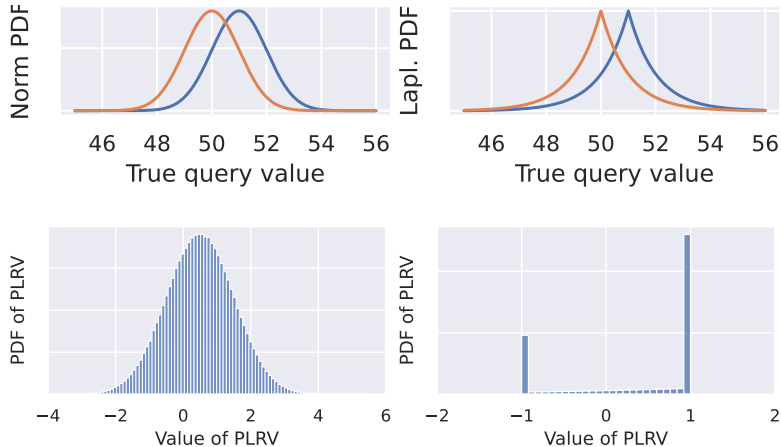
The 'Classical' Gaussian mechanism $\mathcal{M}(D)$ is defined as

$$f(D) + (Y_1, \dots, Y_k)$$

where each Y_n is drawn **independently** from $\mathcal{N}(0, \sigma^2)$, such that

$$\sigma^2 \geq 2 \ln \left(\frac{1.25}{\delta} \right) \frac{(\Delta_2)^2}{\varepsilon^2}$$

Gaussian mechanism is (ϵ, δ) -DP⁴



⁴Non-trivial proof in Appendix A of Dwork and Roth (2013); also note that there are quite a few typos there

General properties of DP algorithms

- 1 Recap
- 2 Using local DP in NLP
- 3 Approximate DP
- 4 Approximate Differential Privacy
- 5 What is this δ doing?
- 6 Gaussian mechanism
- 7 General properties of DP algorithms**

Post-processing

Let $\mathcal{M}(D) \mapsto R$ be a (ϵ, δ) -DP algorithm

Let $f : R \mapsto S$ be an arbitrary (randomized) function

Then $f(\mathcal{M}(D))$ is (ϵ, δ) -DP

In words

Whatever you do with (ϵ, δ) -DP output, you cannot 'weaken' privacy

Group privacy

Let D and D' differ in k positions.

Let $\mathcal{M}(D)$ be (ε, δ) -DP

Then for any output T we have

$$\Pr[\mathcal{M}(D) \in T] \leq \exp(k\varepsilon) \Pr[\mathcal{M}(D') \in T] + k \exp(\varepsilon \cdot (k-1))\delta$$

(this is a loose bound, there are tighter bounds)

Implications for large groups

If k grows, the privacy budget grows exponentially

Basic composition

Let $\mathcal{M} = (M_1, \dots, M_k)$ be a sequence of mechanisms, where each M_i is $(\varepsilon_i, \delta_i)$ -DP. (They might be adaptive)

Then $\mathcal{M}$ is $(\sum_{i=1}^k \varepsilon_i, \sum_{i=1}^k \delta_i)$ -DP

In words

Overall privacy 'budget' can be spent for a sequence of private queries

License and credits

Licensed under Creative Commons
Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)



Credits

Ivan Habernal

Content from ACL Anthology papers licensed under CC-BY

<https://www.aclweb.org/anthology>

Partly inspired by lectures from Antti Honkela, Aurélien Bellet, Gautam Kamath